



Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte/MG | Junho de 2026

Sistemas Operacionais
Professor Ítalo Cunha

Documentação TP3

Implementação null-pointer dereference

Este documento detalha as modificações realizadas no código-fonte do sistema operacional xv6 para a implementação da desreferência de ponteiro nulo

Alunos:

Daniel Vítor Rabelo Rodrigues | Matrícula: 2022043248

Ítalo Leal Lana Santos | Matrícula: 2024013893

Sarah Menks Sperber | Matrícula: 2023001824

1. Visão Geral e Estratégia

No sistema operacional xv6-riscv, em sua configuração original, a memória virtual do processo de usuário é inicializada a partir do endereço virtual `0x0`. O código executável do programa (seções `.text` e `.data`) é carregado logo no início do espaço de endereçamento. Como consequência direta dessa modelagem, a desreferenciação de um ponteiro nulo (`NULL`, que representa o endereço `0`) não causava falhas no sistema; em vez disso, o processo lia ou executava as instruções localizadas na primeira página de código do próprio programa, mascarando erros de programação graves.

O objetivo deste projeto foi alterar a estrutura de gerenciamento de memória virtual e o processo de build do xv6 para impedir qualquer acesso de usuário (leitura, escrita ou execução) ao endereço `0x0`, garantindo que referências a ponteiros nulos resultem em falhas de segmentação (*page faults*) tratadas pelo kernel, que encerra o processo transgressor imediatamente.

A estratégia implementada divide-se em três pilares:

1. Configuração em tempo de compilação (Link-time): Deslocar o início do endereçamento virtual de todos os binários de usuário de `0x0` para `0x1000` (a segunda página de memória virtual, 4096 bytes).

2. Carga segura de executáveis (Exec-time): Modificar o kernel para validar o cabeçalho ELF impedindo que executáveis maliciosos ou corrompidos carreguem código no endereço `0x0`. Adicionalmente, corrigir a lógica de alocação de memória do kernel para evitar o mapeamento implícito da página zero durante a inicialização dos segmentos.

3. Tratamento de exceções (Runtime): Impedir que a página zero seja mapeada em tempo de execução pelo mecanismo de paginação preguiçosa (*lazy allocation*), tratando tentativas de acesso a essa região como violações não recuperáveis.

2. Modificações

2.1. Processo de Build e Configuração do Linker

Os programas de usuário do xv6 são linkados usando um script de linker unificado. Modificamos o arquivo `user/user.ld` para definir que o início da seção de texto ocorra em `0x1000` em vez de `0x0`.

```
--- a/user/user.ld
+++ b/user/user.ld
@@ -2,7 +2,7 @@ OUTPUT_ARCH( "riscv" )
  SECTIONS
  {
-   . = 0x0;
+   . = 0x1000;

    .text : {
        *(.text .text.*)
```

No entanto, o utilitário `forktest` é compilado de forma independente no Makefile, especificando as flags de linker manualmente via linha de comando para gerar um binário reduzido. Atualizamos a flag `-Ttext` de `0` para `0x1000` para manter a paridade na compilação do teste de fork.

```
--- a/Makefile
+++ b/Makefile
@@ -113,7 +113,8 @@ $U/usys.o : $U/usys.S
  $U/_forktest: $U/forktest.o $(ULIB)
    # forktest has less library code linked in - needs to be small
    # in order to be able to max out the proc table.
-   $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o $U/usys.o
+   $(LD) $(LDFLAGS) -N -e main -Ttext 0x1000 -o $U/_forktest $U/forktest.o $U/ulib.o
  $U/usys.o
  $(OBJDUMP) -S $U/_forktest > $U/forktest.asm
```

2.2. Modificações no Kernel para Carregamento e Carga de ELF

A função `kexec` (em `kernel/exec.c`) lê o arquivo ELF executável e inicializa o espaço de endereçamento do processo. Para assegurar que o kernel nunca mapeie o endereço `0x0` para um programa de usuário, foram necessárias duas alterações chaves:

1. Rejeição de `vaddr 0`: Adição de uma validação estrita que impede a carga de qualquer segmento cujo endereço virtual solicitado (`ph.vaddr`) seja `0`.

2. Correção da alocação de memória (Bug do `uvmmalloc`): Na versão original, `kexec` chamava a rotina `uvmmalloc` passando o tamanho do processo acumulado `sz` (que começa em `0` na primeira iteração) como endereço inicial. Ao tentar mapear um segmento que começava em `0x1000`, a chamada original `uvmmalloc(pagetable, 0, 0x1000 + memsz, ...)` acabava alocando e mapeando implicitamente as páginas físicas do intervalo `[0, 0x1000)`. Para corrigir isso, calculamos o endereço inicial exato usando a variável `allocstart`, evitando que o mapeamento espúrio da página zero ocorra.

```
--- a/kernel/exec.c
+++ b/kernel/exec.c
@@ -67,8 +67,11 @@ kexec(char *path, char **argv)
     goto bad;
     if(ph.vaddr % PGSIZE != 0)
         goto bad;
+    if (ph.vaddr == 0)
+        goto bad;
     uint64 sz1;
-    if((sz1 = uvmmalloc(pagetable, sz, ph.vaddr + ph.memsz, flags2perm(ph.flags))) == 0)
+    uint64 allocstart = (ph.vaddr > sz) ? ph.vaddr : sz;
+    if((sz1 = uvmmalloc(pagetable, allocstart, ph.vaddr + ph.memsz, flags2perm(ph.flags))) ==
0)
         goto bad;
     sz = sz1;
```

2.3. Tratamento de Falhas de Página em Tempo de Execução

Como este `xv6` foi modificado para suportar paginação sob demanda (*lazy memory allocation*), quando o processo de usuário tenta ler ou escrever em um endereço não mapeado, a CPU gera um Page Fault que é repassado ao kernel. O tratador original invocava a rotina `vmfault` para instanciar a página física dinamicamente sem checar limites inferiores.

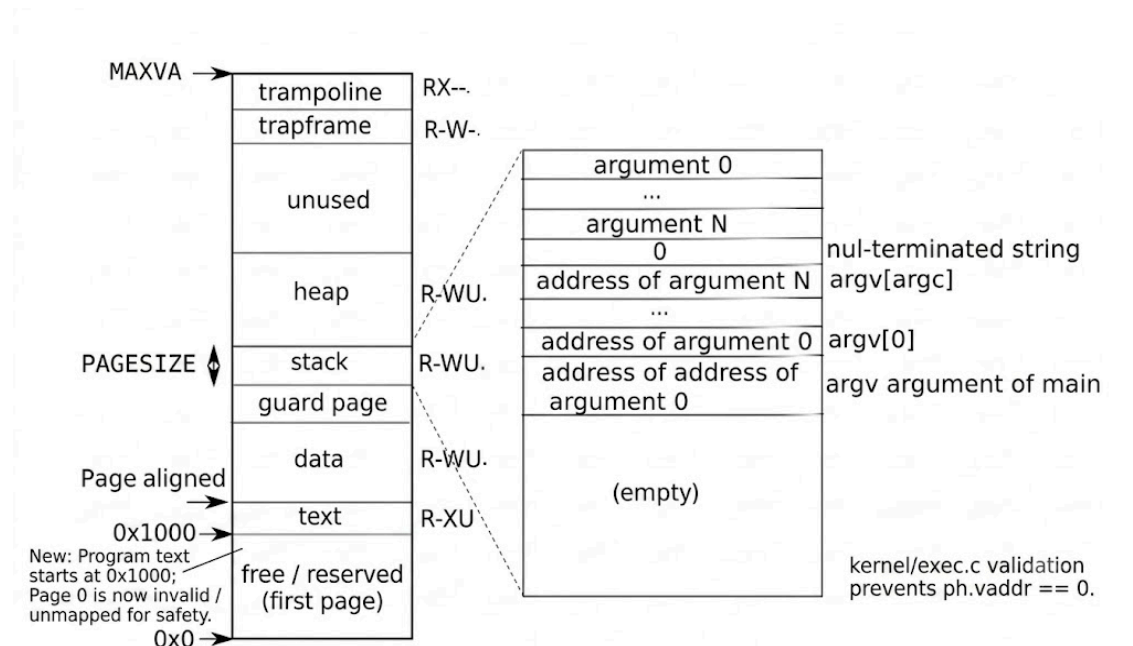
Caso o processo tentasse desreferenciar `NULL`, a rotina de paginação interpretaria o acesso à página `0` como uma falha legítima (pois o endereço `0` estava tecnicamente abaixo do tamanho total do processo `p->sz`), alocando e mascarando a falha. Adicionamos uma barreira impedindo a alocação de qualquer endereço virtual inferior a `PGSIZE` (4096 bytes).

```
--- a/kernel/vm.c
+++ b/kernel/vm.c
@@ -454,7 +454,8 @@ vmfault(pagetable_t pagetable, uint64 va, int read)
{
    uint64 mem;
    struct proc *p = myproc();
-
+    if (va < PGSIZE)
+        return 0;
    if (va >= p->sz)
        return 0;
```

Quando `vmfault` rejeita o acesso (retornando `0`), a falha de página é classificada como não-tratável pela função `usertrap` em `kernel/trap.c`, que sinaliza o encerramento do processo (`setkilled(p)`) e o finaliza via `kexit(-1)`.

3. Espaço de Endereçamento do Processo

Abaixo está a representação gráfica do espaço de endereçamento virtual de um processo de usuário após a aplicação das alterações. Ela estende a estrutura da Figura 3.4 do livro do xv6 adicionando a área reservada/inválida correspondente à primeira página de memória.



- **0x0000 a 0x1000 (Página 0):** Zona inválida. Qualquer tentativa de acesso a esse intervalo de endereços virtuais gera uma interrupção de Page Fault imediata, protegendo o sistema contra ponteiros nulos.
- **0x1000 em diante:** O código executável (.text) e as variáveis globais (.data / .bss) são mapeados sequencialmente. O registrador PC (Program Counter) aponta para o endereço inicial do código de usuário (tipicamente 0x1044 ou 0x1058).
- **Página de Guarda:** Mantida para proteger a pilha contra estouro de limites (stack overflow), localizada logo abaixo da Pilha de Usuário.

4. Testes e validação

Para confirmar o comportamento correto do kernel e afastar regressões, realizamos testes estáticos, testes dinâmicos com instrumentos de depuração e testes de estresse em todo o sistema.

4.1. Código de Teste e Execução (testnullptr)

Criamos o programa user/testnullptr.c que tenta desreferenciar explicitamente o endereço 0:

```
#include "kernel/types.h"
#include "user.h"

int main(void) {
    printf("**Antes do acesso*\n");
    int *p = (int*)0;
    printf("valor de p: %p\n", p);
    printf("desreferenciando p...\n");

    int x = *p;    // Deve causar trap e matar o processo aqui!

    printf("ERRO: nao deveria chegar aqui. valor lido = %d\n", x);
    exit(0);
}
```

Ao executar o binário dentro do xv6, a execução é interrompida exatamente na linha de desreferenciação. O kernel reportou:

```
$ testnullptr
*Antes do acesso*
valor de p: 0x0000000000000000
desreferenciando p...
usertrap(): unexpected scause 0xd pid=3
             sepc=0x102e stval=0x0
$
```

Análise do Log:

- **scause 0xd (13 em decimal):** Exceção correspondente a *Load Page Fault* no RISC-V, comprovando que o processador tentou ler a página virtual e falhou.
- **sepc=0x102e:** O endereço de instrução do erro aponta exatamente dentro do código do programa (0x1000 a 0x2000).
- **stval=0x0:** Indica que a falha de página ocorreu ao tentar traduzir o endereço virtual nulo (0).
- O prompt do terminal (\$) foi imediatamente exibido, confirmando que apenas o processo `testnullptr` foi eliminado, mantendo o sistema em funcionamento estável.

4.2. Inspeção Estática do ELF

Rodamos o utilitário `readelf` para certificar que o processo de build relocou corretamente as seções:

```
$ riscv64-linux-gnu-readelf -l user/_testnullptr
Entry point 0x1044
LOAD ... VirtAddr 0x0000000000001000 ... R E
LOAD ... VirtAddr 0x0000000000002000 ... RW
```

A saída confirma que o primeiro segmento do executável (`LOAD`) foi deslocado para o endereço virtual `0x1000` (texto, leitura e execução) e o segundo segmento para `0x2000` (dados, leitura e escrita), validando a corretude de `user/user.ld`.

4.3. Instrumentação Temporária do Kernel

Para rastrear onde exatamente a página zero continuava mapeada antes da correção final, adicionamos temporariamente um log em `exec.c` chamando a função `walk(p->pagetable, 0, 0)`.

- **Antes da correção em `uvmmalloc`:** A saída do log exibia que o endereço 0 possuía uma PTE válida (`valido=1`) logo após a montagem.
- **Depois da correção em `uvmmalloc`:** O log indicava `valido=0` (sem mapeamento físico), comprovando a resolução do bug de inicialização do executável.

4.4. Testes de Regressão Sistêmicos

Por fim, rodamos o script `./test-xv6.py -q usertests` contendo testes abrangentes de concorrência, fork, manipulação de arquivos e memória virtual. Todos os testes integrados do xv6, incluindo o `forktest` adaptado, passaram com êxito. A desalocação da memória virtual foi confirmada como completa e sem vazamentos, uma vez que a ausência de mapeamentos na página 0 é devidamente tratada pela rotina padrão `uvmmunmap`.

5. Referências

1. **SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G.** *Sistemas Operacionais com Java*. 8. ed. Rio de Janeiro: LTC, 2011.
2. **ARPACI-DUSSEAU, R. H.; ARPACI-DUSSEAU, A. C.** *Operating Systems: Three Easy Pieces*. Virtual Memory Intro (xv6 Projects). Disponível em: <<https://pages.cs.wisc.edu/~remzi/OSTEP/>>. Acesso em: 21 jun. 2026.
3. **COX, R.; KAASHOEK, M. F.; MORRIS, R.** *xv6: a simple, Unix-like teaching operating system*. Massachusetts Institute of Technology (MIT), 2022. Disponível em: <<https://pdos.csail.mit.edu/6.828/2022/xv6/book-riscv-rev3.pdf>>. Acesso em: 18 jun. 2026.
4. **STACKOVERFLOW.** *Adding illegal null-dereference to xv6*. Disponível em: <<https://stackoverflow.com/questions/69607698/adding-illegal-null-dereference-to-xv6>>. Acesso em: 18 jun. 2026.